Ajout d'un praticien

1. Détail de la réalisation

Il faut créer un déclencheur avantAjoutPraticien chargé de contrôler le respect de l'ensemble des contraintes
Il faut créer une procédure stockée pour ajouter un praticien et compléter la méthode l'utilisant dans la classe Passerelle.
Il faut créer un formulaire et réaliser son ouverture depuis le formulaire FrmBase

2. Les triggers

Le déclencheur avantAjoutPraticien doit réaliser les actions suivantes :

Il vérifie que :

le nom doit être composé de 3 à 25 caractères
le prénom doit être composé de 3 à 30 caractères,
nom et prénom se composent de groupes de lettres éventuellement accentuées, séparés par un espace, ou une apostrophe ou un tiret'
le couple code postal/ville correspond à une ville gérée par le visiteur
le type de praticien est renseigné et il existe bien dans la table typepraticien,
l'email et le téléphone sont renseignés, ils respectent leur format et sont uniques dans la table praticien,
la spécialité (si renseignée) existe bien dans la table specialite.

Il met en forme le nom et le prénom à l'aide de la fonction stockée miseEnFormeNomPrenom(x)

exemple : set new.nom = miseEnFormeNomPrenom(new.nom);

3. Les procédures stockées

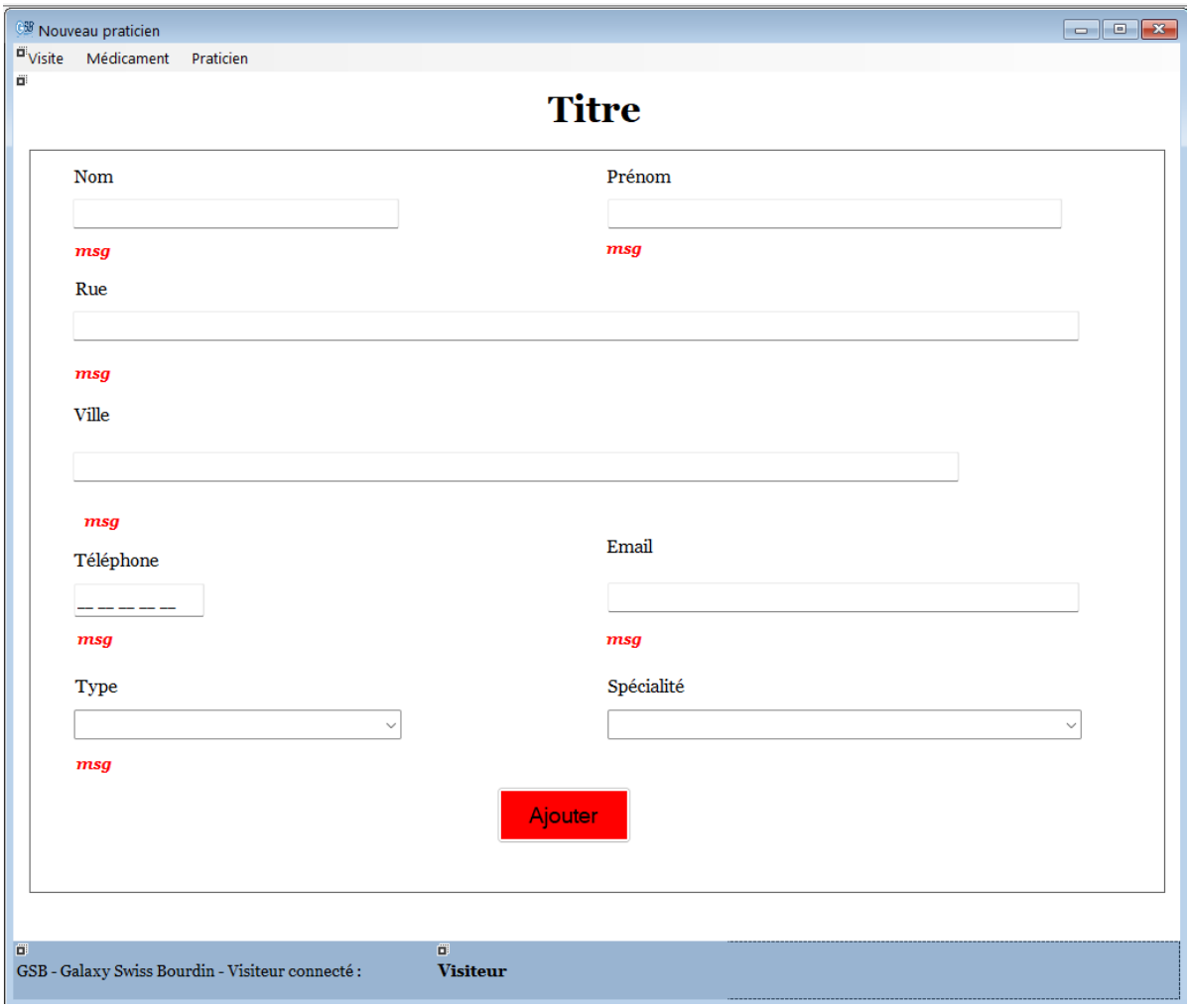
La procédure stockée ajouterPraticien doit simplement exécuter une requête insert sur la table praticien en transmettant dans ses paramètres les valeurs des champs nom, prenom, rue, codePostal, ville, telephone, email, idType et idSpecialite

4. La méthode ajouterPraticien

Elle appelle la procédure stockée ajouterPraticien en intégrant dans son dictionnaire de paramètres (Parameters) les valeurs transmises en paramètre

Elle retourne la valeur de l'identifiant du praticien (auto-increment) en utilisant la propriété LastInsertedId de l'objet MySQLCommand (cmd)
.

5. Proposition d'interface graphique



Principaux composants

panel : panelCentral. Centré en appelant la méthode centrerFormulaire();
label pour les messages d'erreur en rouge : messageNom, messagePrenom, etc.
textBox pour les champs de saisie : txtNom, txtPrenom, txtRue, txtVille, txtEmail
MaskedTextBox avec le masque 00 00 00 00 00 00 (propriété mask) pour le téléphone :mtbTelephone
ComboBox pour type (cbxType) et pour Spécialité (cbxSpecialite)
Button pour le bouton Ajouter (btnAjouter)

Principales méthodes

parametrerComposant()
ajout() et ajouter()
controlerVille(), controlerEmail(), controlerTelephone() et controlerType()

Modification d'un praticien

La méthode parametrerComposant doit paramétrer les zones de liste et mettre en place l'autocomplétion sur la ville afin de limiter les valeurs possibles

```
txtVille.AutoCompleteMode = AutoCompleteMode.SuggestAppend;
 txtVille.AutoCompleteSource = AutoCompleteSource.CustomSource;
 var source = new AutoCompleteStringCollection();
 foreach (Ville ville in session!.MesVilles)  {
    source.Add(ville.Nom);
 }
 txtVille.AutoCompleteCustomSource = source;
```

Cela permettra ensuite à la méthode ajouter de récupérer automatiquement le code postal correspondant à la ville saisie

```
Ville laVille = session!.MesVilles.Find(x => x.Nom == txtVille.Text)!;
string codePostal = laVille.Code;
```

La méthode controlerVille doit vérifier que le champ txtVille est renseigné et que sa valeur correspond bien à une ville contenue dans la collection MesVilles de l'objet session.
Elle alimente le message d'erreur associé et retourner vvrai ou faux.

En utilisant un champ MaskedTextBox pour le téléphone, la méthode controlerTelephone doit simplement vérifier que le marque est rempli

```
private bool controlerTelephone() {
   if (!mtbTelephone.MaskFull)   {
      messageTelephone.Text = "Le téléphone du praticien doit être précisé";
      messageTelephone.Visible = true;
      return false;
   }
   messageTelephone.Visible = false; // cacher le message si tout est bon
   return true;
}
```

La méthode controlerEmail doit vérifier en utilisant une expression régulière la validité de l'adresse

```
private bool controlerEmail()  {
    if (txtEmail.Text == string.Empty)      {
       messageEmail.Text = "L'adresse mail du praticien doit être précisée";
       messageEmail.Visible = true;
       return false;
    }
    Regex uneExpression = new Regex(@"….");
    if (!uneExpression.IsMatch(txtEmail.Text)) {
       messageEmail.Text = "L'adresse mail n'est pas valide";
       messageEmail.Visible = true;
       return false;
    }
    messageEmail.Text = "";
    messageEmail.Visible = false;
    return true;
 }
```

Modification d'un praticien

La méthode controlerType() doit vérifier que le type est bien sélectionné :

```
private bool controlerType()  {
    if (cbxType.SelectedIndex == -1)   {
        messageType.Text = "Veuillez sélectionner le type de praticien.";
        messageType.Visible = true;
        return false;
    }
    messageType.Text = "";
    messageType.Visible = false;
    return true;
}
```

Pour les autres champs de saisie (le nom, le prénom, et la rue), vous pouvez vous contenter de vérifier simplement que le champ de saisie est bien renseigné en utilisant cette méthode générique

```
private bool controlerChamp(TextBox txt, Label lblMessage, string message)   {
    if (string.IsNullOrWhiteSpace(txt.Text))       {
        lblMessage.Text = message;
        lblMessage.Visible = true;
        return false;
    }
    lblMessage.Text = "";
    lblMessage.Visible = false;
    return true;
}
```

La méthode ajout() attachée au bouton 'ajouter' centralise l'ensemble des contrôles avant de lancer la méthode ajouter

```
private void ajout()  {
    // pas de contrôle sur les zone de liste qui peuvent rester vide
    bool nomOk = controlerChamp(txtNom, messageNom, "Le nom du praticien doit être précisé");
    bool prenomOk = controlerChamp(txtPrenom, messagePrenom, "Le prénom du praticien doit être
précisé");
    bool rueOk = controlerChamp(txtRue, messageRue, "La rue du praticien doit être précisée");
    bool villeOk = controlerVille();
    bool emailOk = controlerEmail();
    bool telephoneOk = controlerTelephone();
    bool typeOk = controlerType();
    if (nomOk && prenomOk && rueOk && villeOk && emailOk && telephoneOk && typeOk)  {
        ajouter();
    }
}
```

Modification d'un praticien

La méthode ajouter() doit :

- Récupérer toutes les valeurs à transmettre à la méthode ajouterPraticien de la classe Passerelle
- Appeler la méthode ajouterPraticien en récupérant l'identifiant généré
- Mettre à jour l'objet session
- Mettre à jour l'interface (vider les champs de saisie et afficher un message de réussite)

Déjà indiquée plus haut, la récupération du code Postal s'effectue par l'intermédiaire du champ txtVille et de la collection MesVilles de l'objet session

La récupération de l'id du type s'effectue par l'intermédiaire de la zone de liste cbxType

```
string idType =  ((TypePraticien) cbxType.SelectedItem!).Id;
```

Même chose pour idSpecialite mais seulement si la zone de liste cbxSpecialite est renseignée
Si elle n'est pas renseignée, la valeur 'null' est attribuée

La mise à jour de l'objet session consiste à ajouter ce nouveau praticien dans la collection MesPraticiens et ordonner la collection.

Pour cela il faut créer un objet Praticien ce qui demande préalablement de récupérer l'objet TypePraticien attaché et éventuellement l'objet Specialite.

Cela peut se faire en les recherchant à l'aide de la méthode Find sur les collection correspondante LesTypesPraticien et LesSpecialites.

Toutes les erreurs SQL sont capturées et affichées via MessageBox pour informer l'utilisateur